

Homework 4

601.482/682 Deep Learning

Fall 2025

September 24, 2025

Due Wednesday, October 8, 2025, 11:59 pm EST

Submission Instructions. Please submit the following **two items** to Gradescope (entry code NYY4V7):

1. Your report (LaTeX generated PDF) named `homework4-report.pdf`.
2. A Zip file containing: (1) Your Python Jupyter Notebook (`homework4-notebook.ipynb`) and (2) an exported PDF of the notebook (with all cell outputs) to `homework4-notebook.pdf`.
 - * On Google Colab, you may use “File” - “Print” - Save as PDF.
 - * On Visual Studio, you may use “Jupyter: Export to PDF” in the Command palette.

Note 1) This is the first assignment that it is important to be cognizant of model training times. The MLP models you design in Problem 1 likely won’t take longer than a few minutes to train at maximum. However, in Problem 2, you will implement two convolutional neural networks *and* experiment with hyperparameters/architectures. For context, our proposed designs required 40-60 seconds per epoch, and we ask you to train your models for at least 50 epochs. We recommend you design and test your model at a small scale (e.g. 5-10 epochs) before scaling up for the final run.

Note 2) You will be using PyTorch for the majority of this assignment. We **strongly recommend** using Google Colaboratory (<https://colab.research.google.com>) as well, though you are welcome to work locally if desired.

1. We learned that a two-layer MLP can model polygonal decision boundaries. You are given data sampled from a “two-pentagons” decision boundary. Data points within the pentagons are considered “true” (labeled as 1) while data points outside are considered “false” (labeled as 0).

Please load the data via `numpy.load()` function and check that you obtain a numpy array with a shape of (60000,3). The first two columns are coordinates (x, y) and the third column represents the labels.

The vertices of the 2 polygons are (500, 1000), (300, 800), (400, 600), (600, 600), (700, 800), and (500, 600), (100, 400), (300, 200), (700, 200), (900, 400). We have provided some visualization functions in the Jupyter notebook. The decision boundary can be modeled with a total of $2 \times 5 + 2 + 1$ neurons with threshold activation functions.

- (a) We have set up an MLP with threshold activation by analytically obtaining weights that can model the above decision boundary.
 - i. Implement the “AND gate” and “OR gate” to predict all points within the 2 polygons as the positive class. Use the `predict_output_v1()` to test your implementations. Attach the visualization to your report.

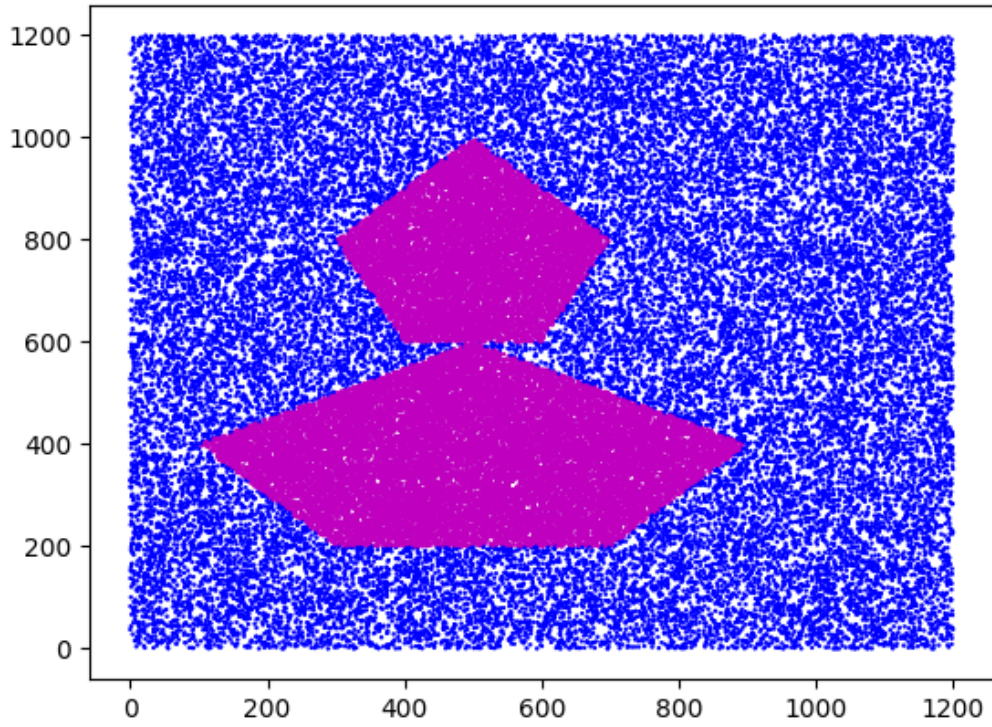


Figure 1: output of v1

- ii. Modify the code in `predict_output_v2()` to predict only the points in polygon 1 as the positive class. Attach the visualization to your report. *Note: You should only need to update the application of the gates themselves.*

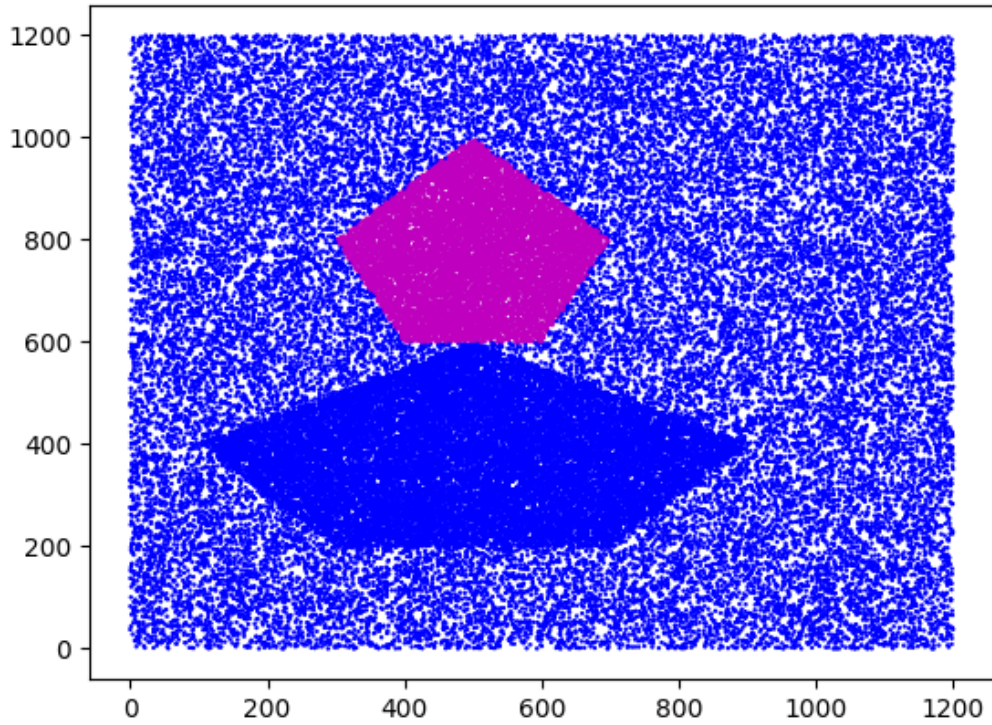


Figure 2: output of v2

Note: We reverse-engineered the weights from the known decision boundaries but in the

real world, we do not know the decision boundaries. In a more realistic scenario, we would thus need to discover (learn) the decision boundaries from training data which we will explore next.

- (b) Build an MLP using sigmoid activation functions to replace the “AND/OR” gates. You are required to use PyTorch for this question and all remaining parts of the assignment. We **strongly recommend using Google Colaboratory** for ease of debugging and modeling flexibility. Whenever we provide code in subsequent assignments, it will be optimized for this setup.¹

Model hyperparameters:

- i. Use 2 hidden layers, and 1 output layer with binary-cross-entropy loss ²
- ii. Use 10 nodes in the first hidden layer, 2 nodes in the second hidden layer, and 1 output node (binary classification), which mirrors the capacity of the model in 1a)
- iii. Initialize the weights using Xavier Initialization
- iv. You may use any combination of batch size, training epochs, and learning rate. (Note that a larger batch size typically corresponds to a larger learning rate)
- v. Do not use any regularization for this problem

Use gradient descent to optimize the parameters. You should expect an average accuracy greater than 90%. **Please use the first 50000 points as your training set and the remaining 10000 as your test set. You may also want to normalize (i.e. center and scale) the data before training.**

TODO:

- i. Train your model using 5 random seeds and report the mean and standard deviation of the train and test accuracy at the end of 500 epochs. Attach the plots of training loss and testing accuracy change over epochs.

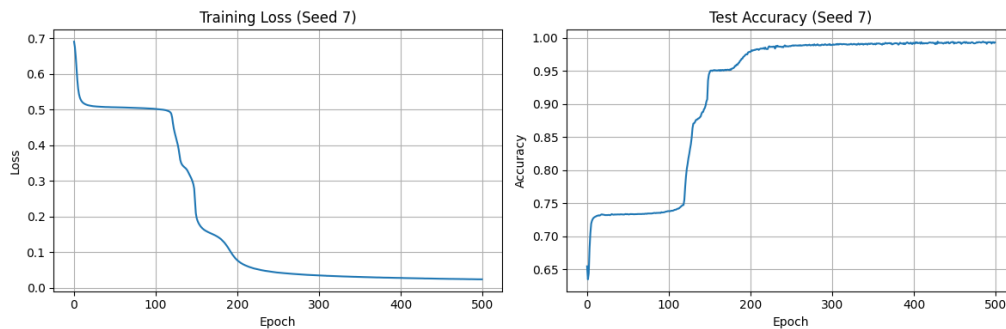


Figure 3: Training loss and test accuracy for seed 7

¹<https://pytorch.org/>. If you have not programmed with PyTorch before, give yourself some time to go through some online tutorials. There are many on the internet so it is hard to recommend a specific one, it would really depend on your level of comfort. You may also find it helpful to review the PyTorch resources on Piazza (<https://piazza.com/jhu/fall12025/cs482682/resources>).

²See discussion here <https://stackoverflow.com/questions/53628622/loss-function-its-inputs-for-binary-classification-pytorch>

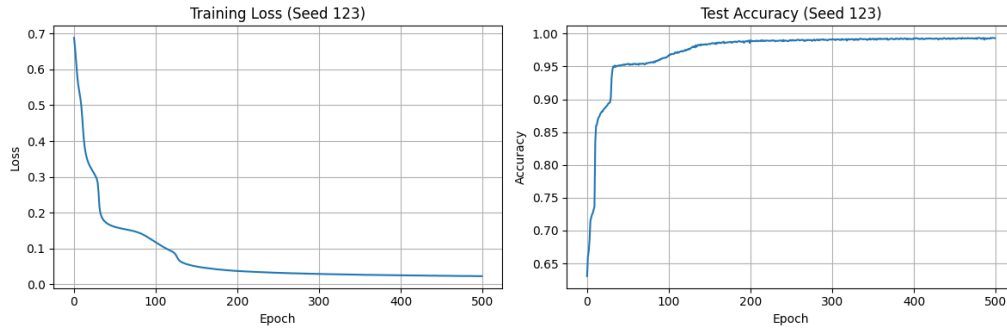


Figure 4: Training loss and test accuracy for seed 123

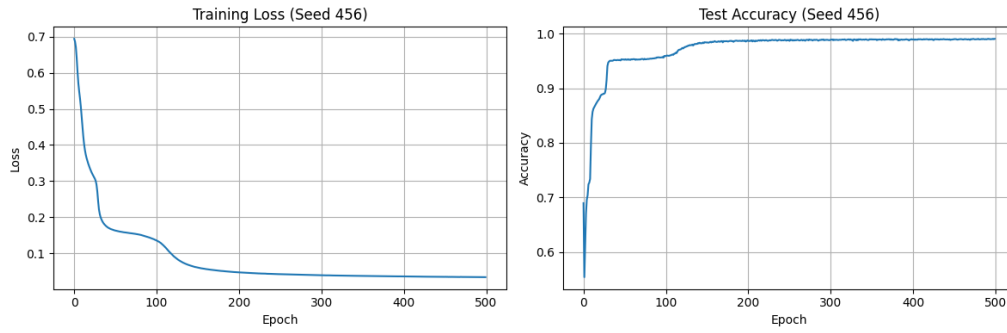


Figure 5: Training loss and test accuracy for seed 456

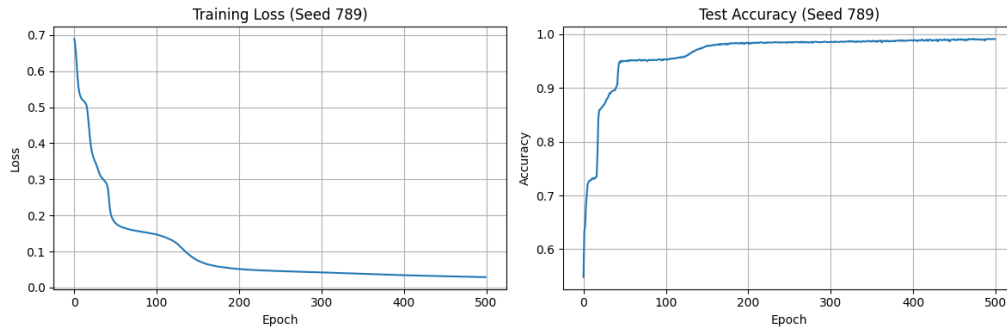


Figure 6: Training loss and test accuracy for seed 789

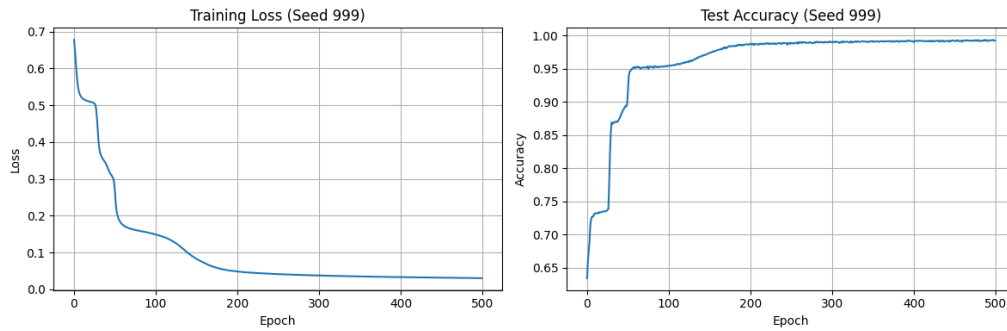


Figure 7: Training loss and test accuracy for seed 999

- ii. In your report, attach the visualization of the test set prediction (i.e. polygon figure).

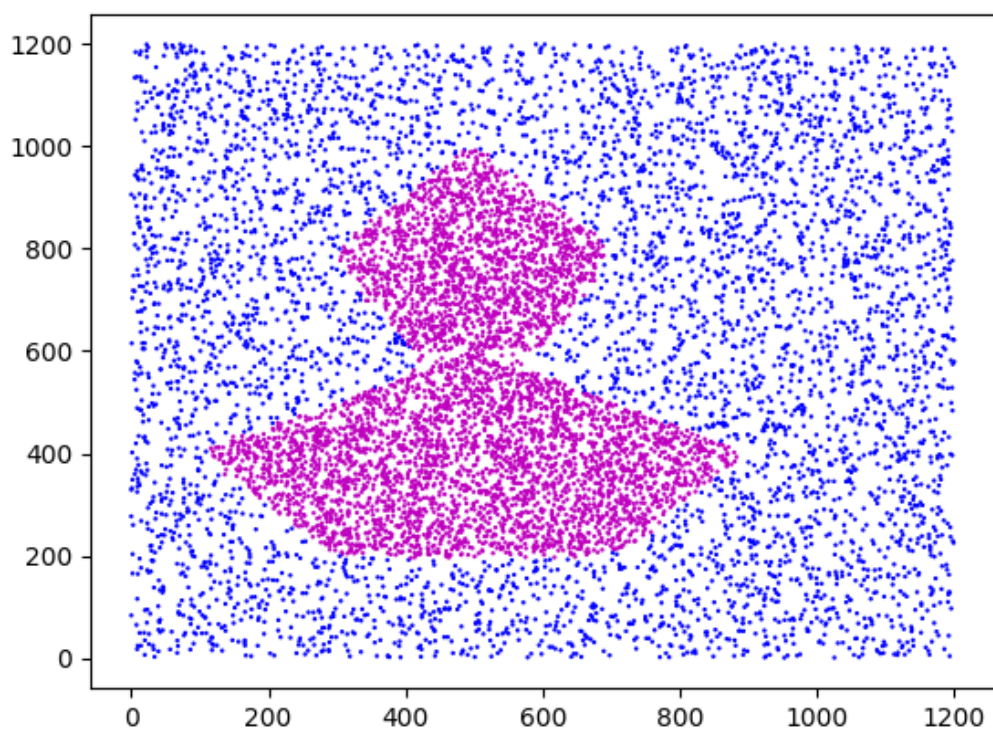


Figure 8: Visualization for seed 7

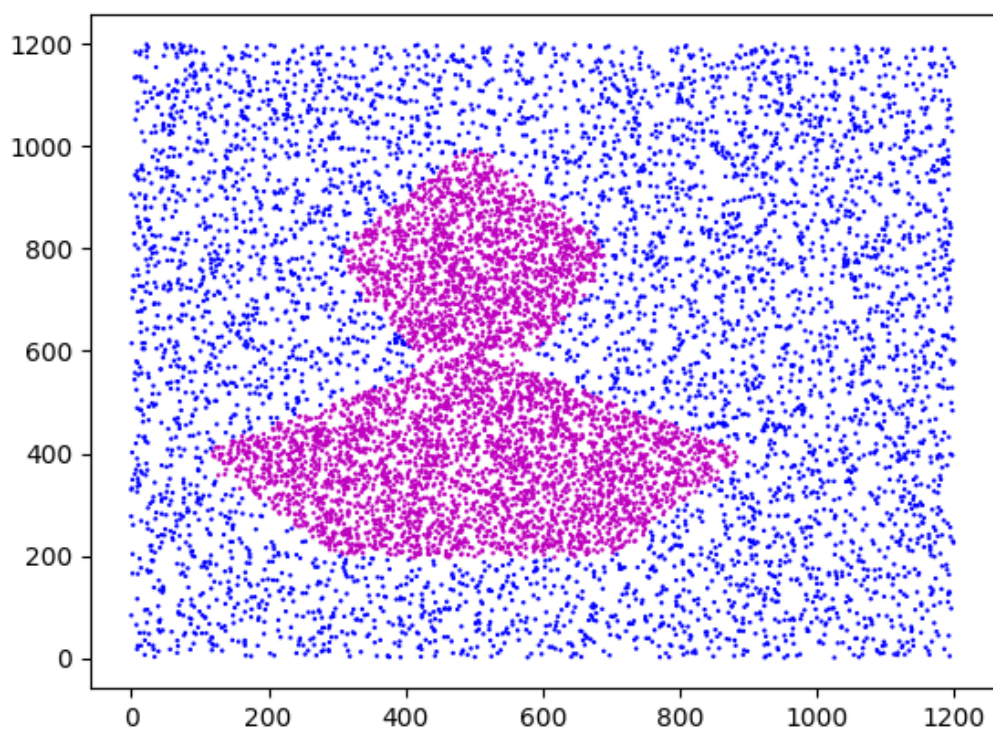


Figure 9: Visualization for seed 123

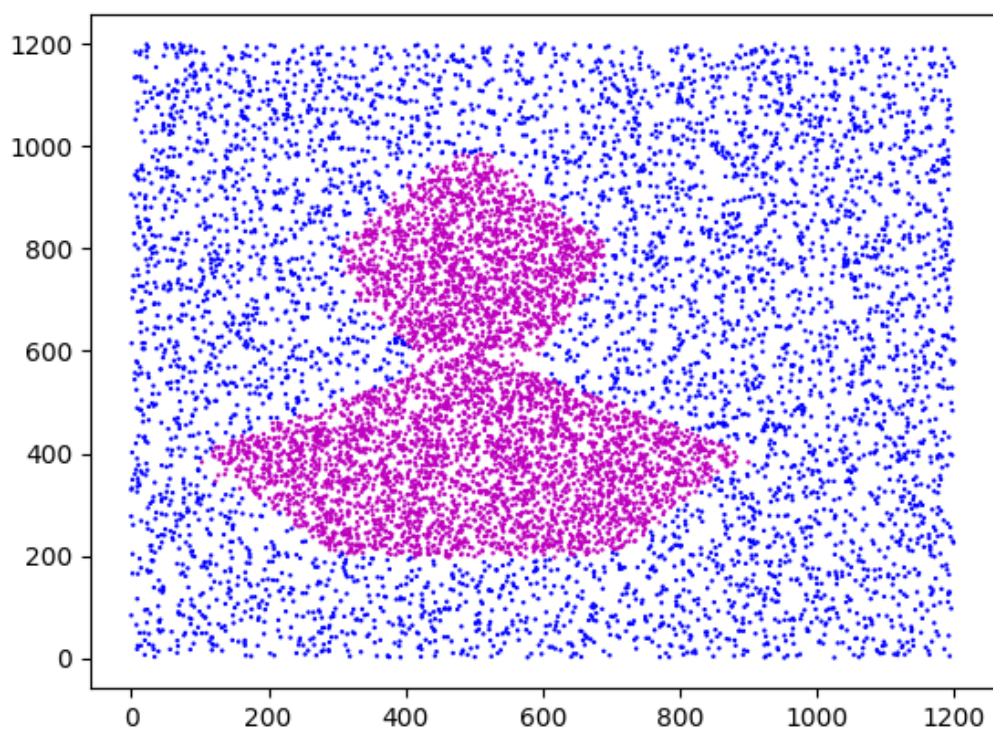


Figure 10: Visualization for seed 456

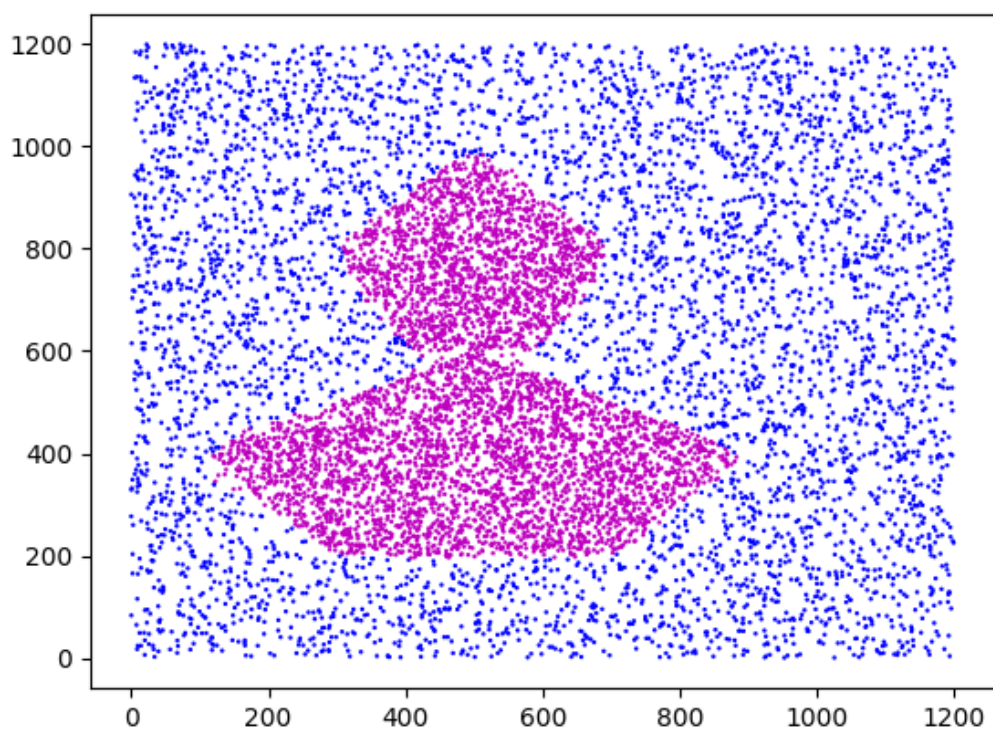


Figure 11: Visualization for seed 789

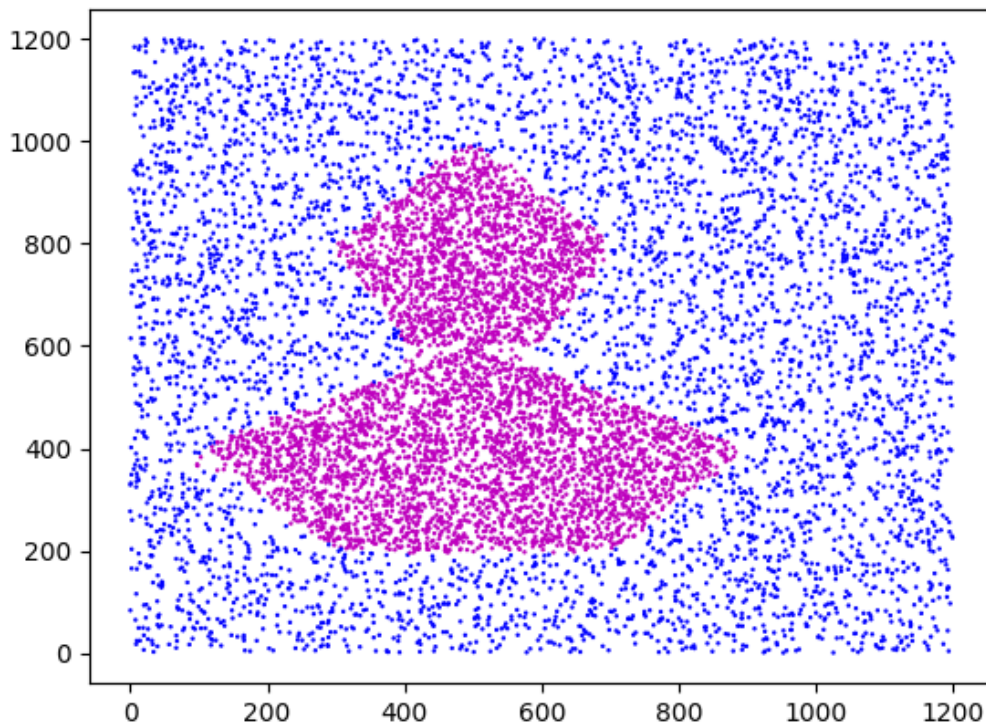


Figure 12: Visualization for seed 999

- iii. Brief discussion. Are you able to find a solution that performs almost as well as the “manual” solution in (a)?

Summary Statistics

Train Accuracies: ['0.9920', '0.9915', '0.9896', '0.9894', '0.9915']

Test Accuracies: ['0.9928', '0.9929', '0.9905', '0.9907', '0.9924']

Mean Train Accuracy: 0.9908 ± 0.0011

Mean Test Accuracy: 0.9919 ± 0.0010

The best test accuracy was 0.9929 with seed 123. This is pretty close to the manual solution in (a).

- (c) Build an MLP with a larger capacity (increase the depth and width). What do you expect to happen to your model’s ability to learn the appropriate decision boundaries when you increase the depth? Increase the width? *Note: Since this is a toy dataset, you might not actually observe differences in performance compared to your 1b solution. However, you should be able to reason about expected behavior based on our lecture discussions.*
- Like before, train your model using 5 random seeds and report the mean and standard deviation of the train and test accuracy at the end of 500 epochs.
 - Report what depth and width you used. Briefly discuss what you notice about your model performance as you change these architecture decisions. *Hint: Think about our discussions from the lecture regarding model capacity.*

2 hidden layers with increased width: [128, 64]

Increased Width Summary:

Train Accuracies: ['0.9925', '0.9927', '0.9930', '0.9934', '0.9929']

Test Accuracies: ['0.9927', '0.9933', '0.9939', '0.9942', '0.9934']

Mean Train Accuracy: 0.9929 ± 0.0003

Mean Test Accuracy: 0.9935 ± 0.0005

4 hidden layers with similar capacity: [32, 16, 8, 4]

Increased Depth Summary:

Train Accuracies: ['0.9923', '0.9938', '0.9833', '0.9947', '0.9936']

Test Accuracies: ['0.9922', '0.9948', '0.9853', '0.9955', '0.9944']

Mean Train Accuracy: 0.9915 ± 0.0042

Mean Test Accuracy: 0.9924 ± 0.0037

4 hidden layers with increased width as well: [256, 128, 64, 32]

Increased Width and Depth Summary:

Train Accuracies: ['0.9927', '0.9947', '0.9949', '0.9943', '0.9939']

Test Accuracies: ['0.9932', '0.9954', '0.9954', '0.9942', '0.9945']

Mean Train Accuracy: 0.9941 ± 0.0008

Mean Test Accuracy: 0.9945 ± 0.0008

What I notice about model performance:

Increased Width (2 layers) vs Increased Depth (4 layers): The wider network achieved more consistent performance (lower standard deviation) compared to the deeper but narrower network. This suggests that for this problem, width may be more beneficial than depth alone.

Best Performance: The model with both increased depth AND width (4 layers, [256, 128, 64, 32]) achieved the highest accuracy and most stable performance. This aligns with model capacity theory: higher capacity models can learn more complex representations when sufficient data is available.

Overfitting Observations: All models show good generalization with test accuracy comparable to or slightly better than training accuracy, indicating no significant overfitting. The increased depth model showed more variance in performance across runs, suggesting potential optimization challenges.

This pattern reflects the fundamental trade-off in neural architecture design: sufficient model capacity is necessary to learn complex patterns, but must be balanced with the risk of overfitting and training instability.

- iii. Attach the plots of training loss and testing accuracy change over epochs.

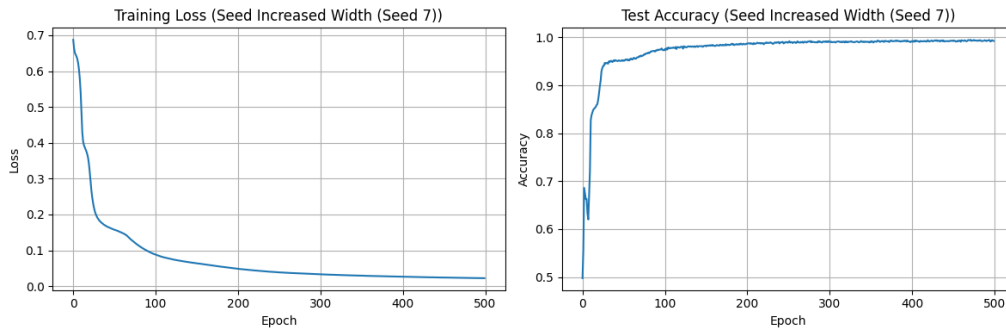


Figure 13: Training loss and test accuracy for increased width, seed 7

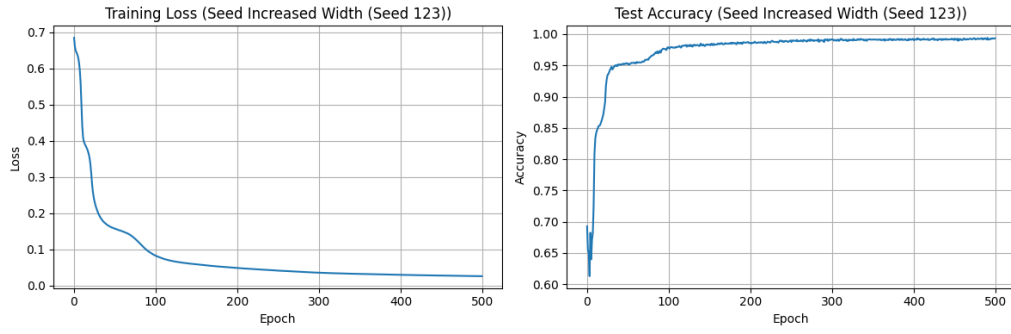


Figure 14: Training loss and test accuracy for increased width, seed 123

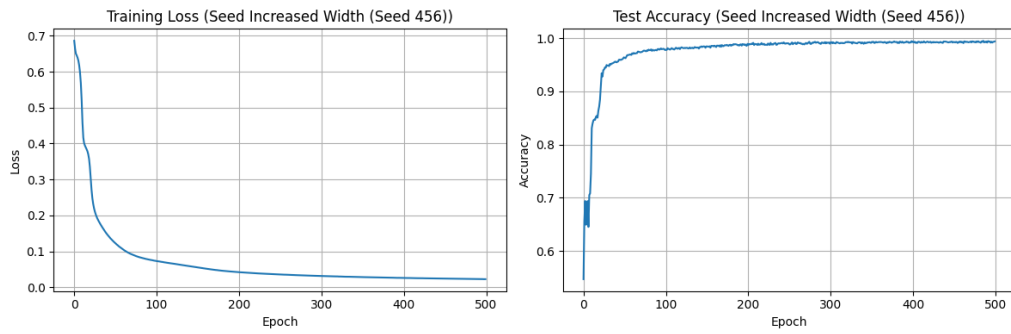


Figure 15: Training loss and test accuracy for increased width, seed 456

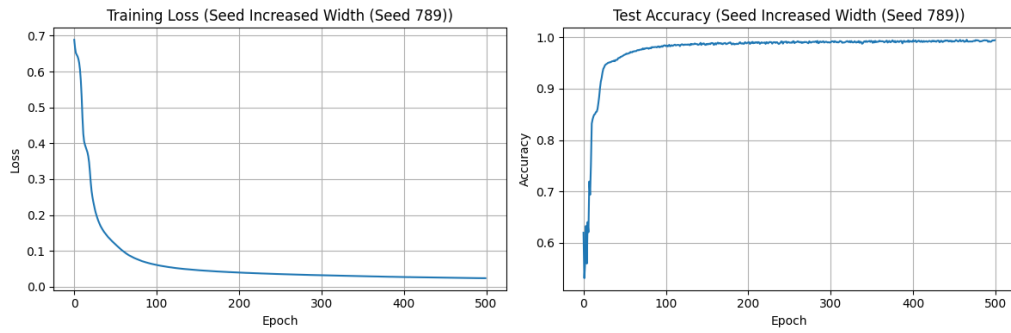


Figure 16: Training loss and test accuracy for increased width, seed 789

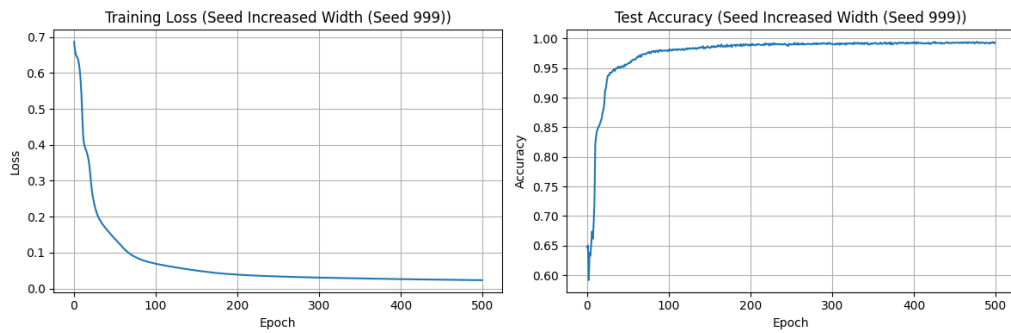


Figure 17: Training loss and test accuracy for increased width, seed 999

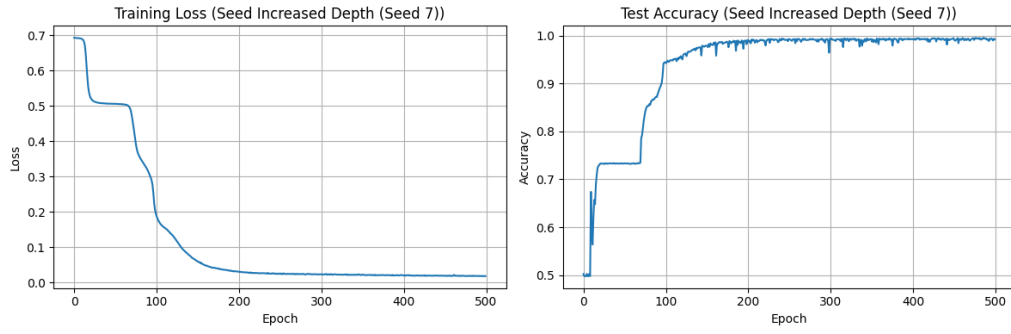


Figure 18: Training loss and test accuracy for increased depth, seed 7

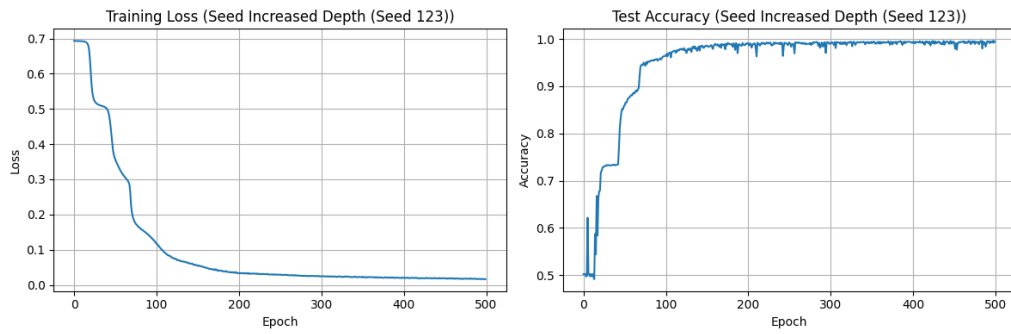


Figure 19: Training loss and test accuracy for increased depth, seed 123

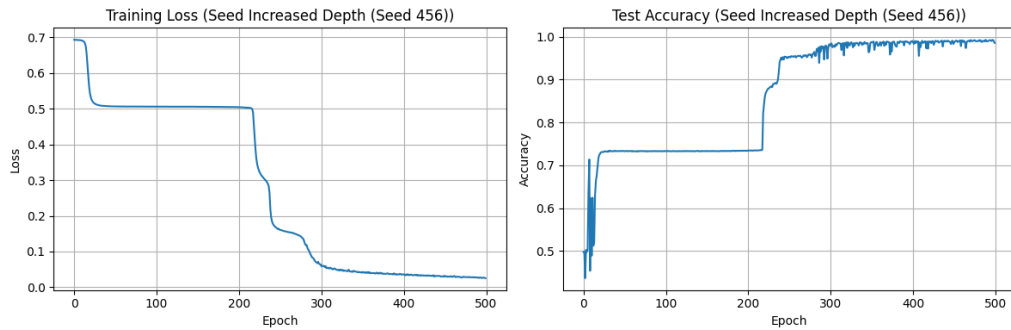


Figure 20: Training loss and test accuracy for increased depth, seed 456

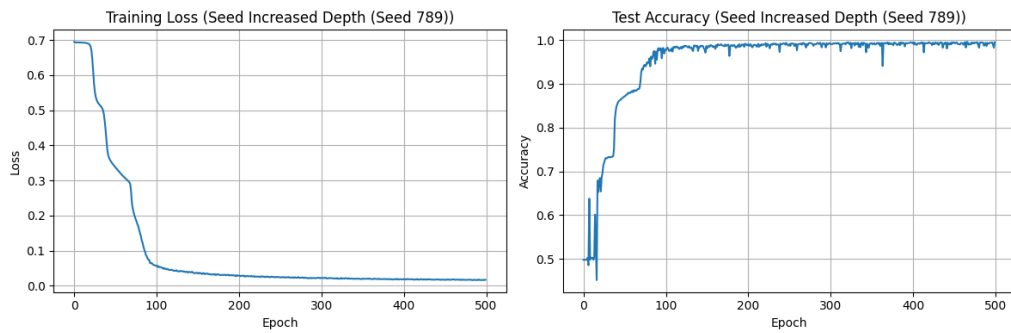


Figure 21: Training loss and test accuracy for increased depth, seed 789

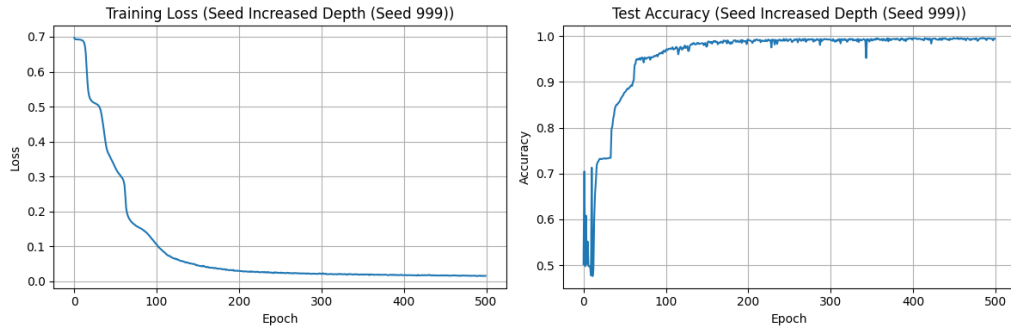


Figure 22: Training loss and test accuracy for increased depth, seed 999

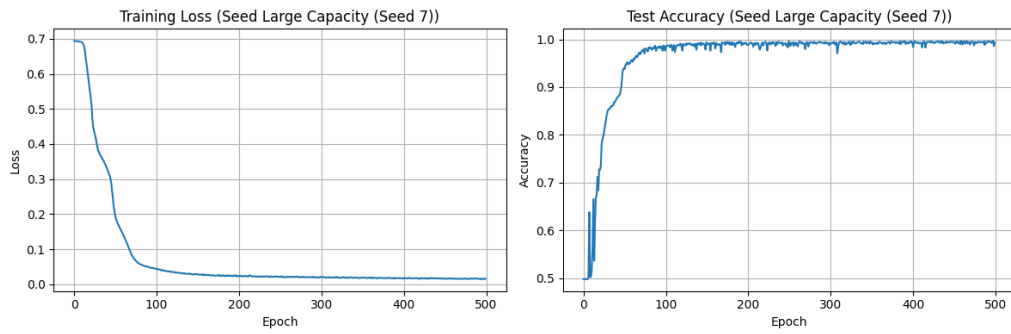


Figure 23: Training loss and test accuracy for increased width and depth, seed 7

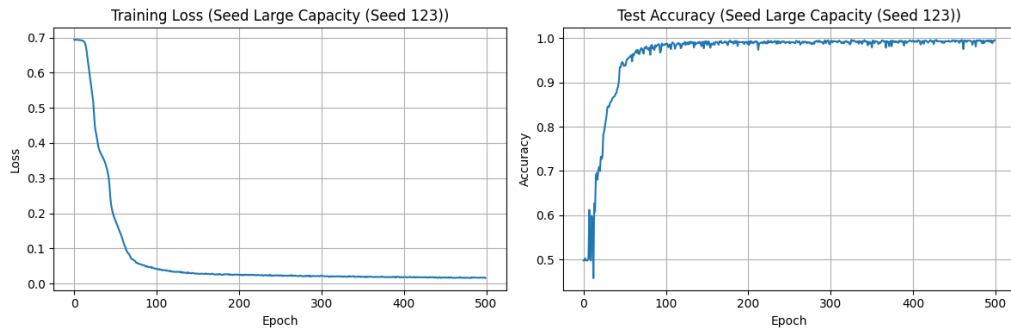


Figure 24: Training loss and test accuracy for increased width and depth, seed 123

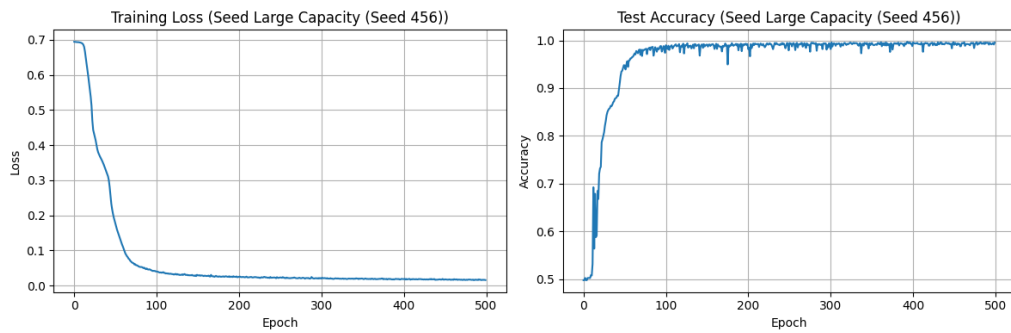


Figure 25: Training loss and test accuracy for increased width and depth, seed 456

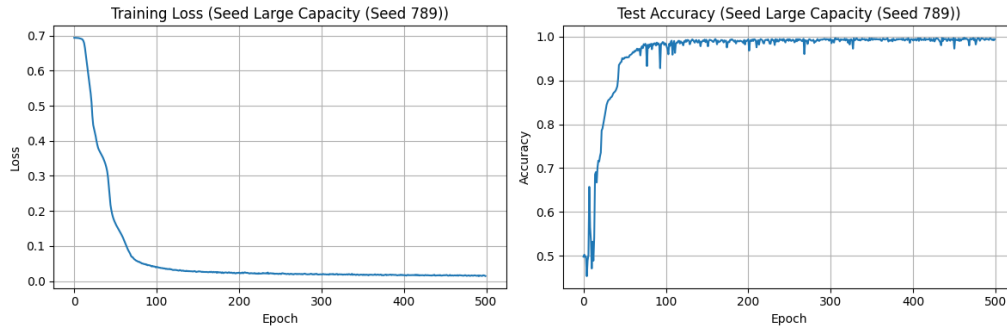


Figure 26: Training loss and test accuracy for increased width and depth, seed 789

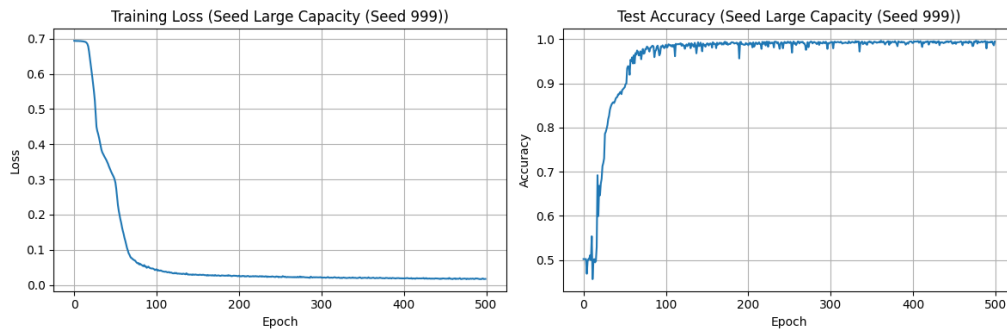


Figure 27: Training loss and test accuracy for increased width and depth, seed 999

- iv. Attach the visualization of the prediction for the test set (i.e. polygon figure).

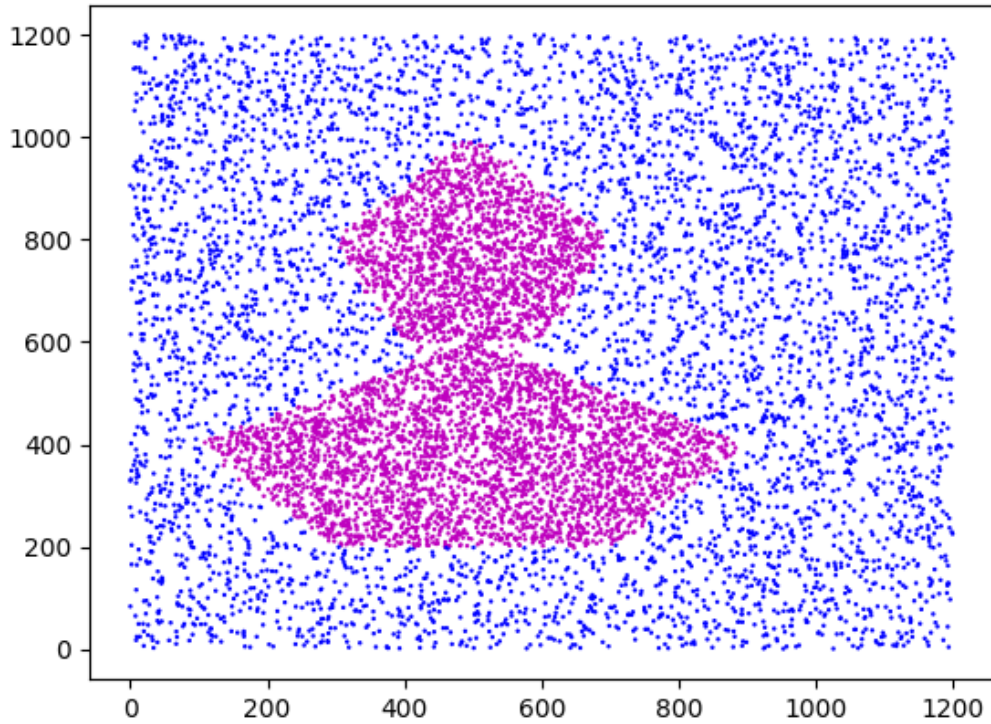


Figure 28: Visualization for increased width, seed 999

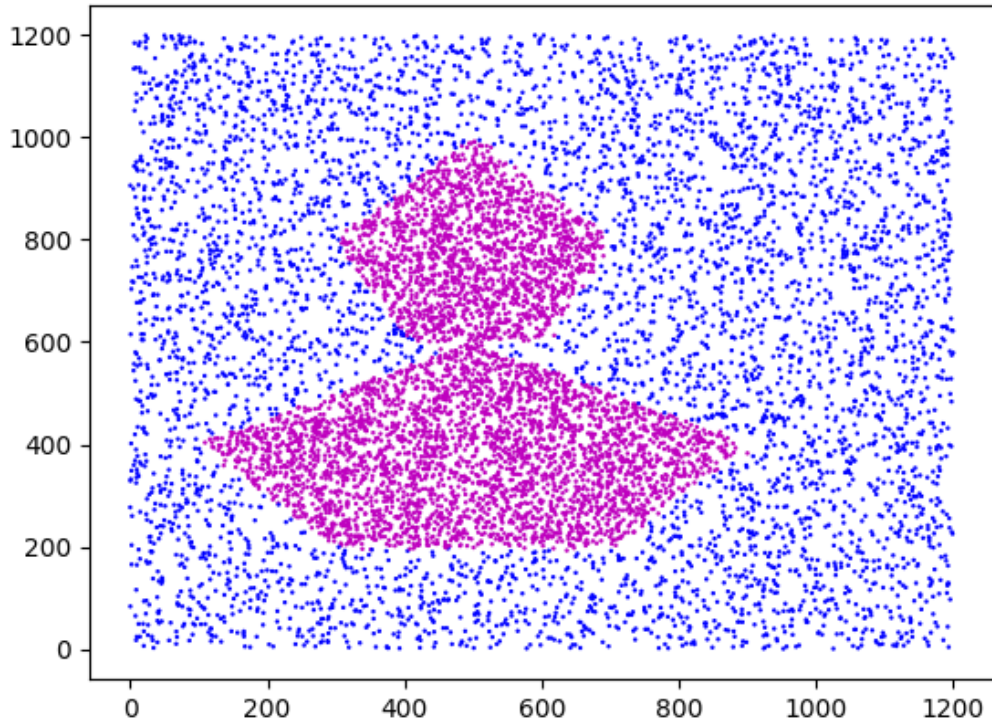


Figure 29: Visualization for increased depth, seed 999

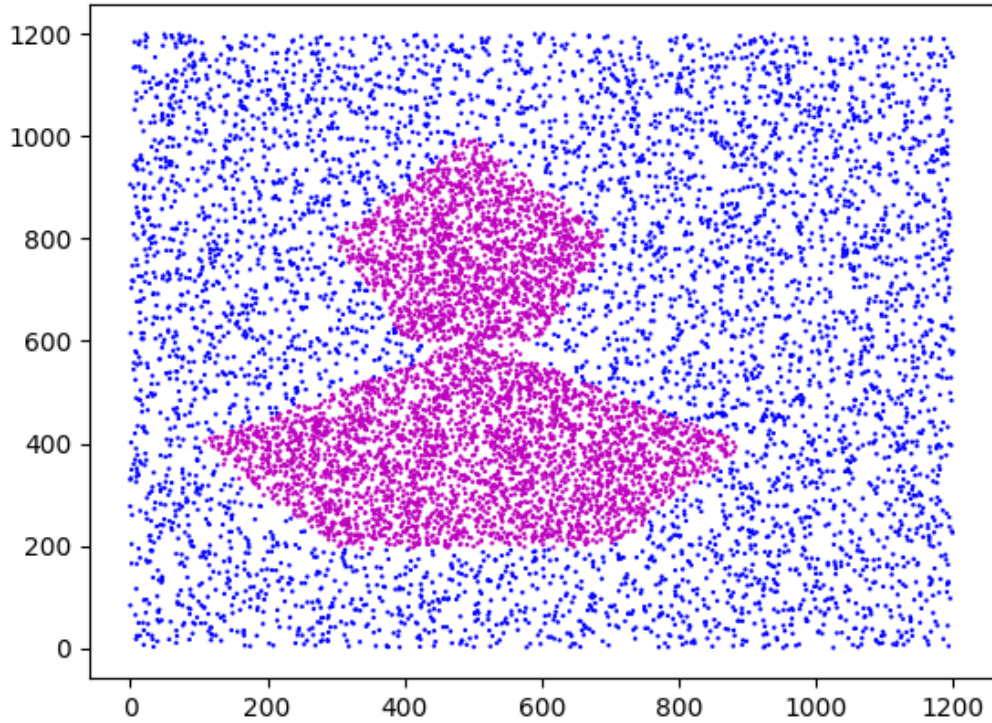


Figure 30: Visualization for increased width and depth, seed 999

- (d) In part (b) we had fixed the hyperparameter choices, but in part (c) you adjusted those choices for the MLP in terms of depth and width of the network. Would you consider your test results to be valid and generalizable? Please briefly discuss.

For Part (b): The test results are considered valid and generalizable. This is because all model specifications (like the number of layers and nodes) were predetermined and fixed before any evaluation. Since the test set was not used to make any decisions about the model, it serves as an unbiased benchmark for performance.

For Part (c): The test results are not considered valid or generalizable. The act of building models with different capacities (more depth, more width) and then comparing their results on the test set fundamentally changes the role of that data. It is no longer just a final test, it becomes a tool for model selection. This introduces a form of data leakage, where information from the test set indirectly influences the design of the model. Consequently, the reported test accuracy becomes an over-optimistic measure of how the model would perform on truly unseen, real-world data.

2. In this problem, we will explore convolutional neural networks (CNNs). We will be working with the FashionMNIST dataset.³ This dataset only has a train-test split, therefore we will be using the last 10000 training instances as the validation set. Again, we will use PyTorch⁴ for this assignment. Because training times can be quite long, you *do not* need to train your model with multiple random seeds — 1 is enough, but of course feel free to expand if you have time and are confident in your model. You should try to train your models for at least 50 epochs.

- (a) Design a small CNN (e.g. 3-4 hidden layers) using the tools we learned in class such as
- convolution and pooling layers
 - activation functions (other than sigmoid)

TODO:

- Briefly** describe your architecture in your report and explain your design choices (e.g. I used ReLU activation because...)

Please design the network architecture by yourself. This is not about performance.

I designed a sequential CNN with three convolutional blocks followed by a classifier. Each block uses Conv2d → ReLU → MaxPool with increasing filters (32→64→128). The final classifier has a 256-unit dense layer before the 10-class output.

Design Choices

ReLU activation for computational efficiency and mitigation of vanishing gradients
Progressive filter doubling (32→64→128) to capture hierarchical features while managing parameters

MaxPooling after each block for spatial hierarchy and translation invariance

CrossEntropyLoss as standard for multi-class classification
SGD optimizer for stable, controlled optimization.

Tried too learn of learning rates (i.e. 0.001) and ended up with test accuracy around 83%. So I increased the learning rate to 0.01.

- Report the train, validation, and test accuracy of your best model (e.g. if the best model was obtained at epoch 50, report the train, validation, and test accuracy at epoch 50).

Epoch 50/50: Train Accuracy: 93.83%, Val Accuracy: 91.10%
Final Test Accuracy: 90.54%

- Attach the plots of training loss and validation accuracy change over epochs.

³The full FashionMNIST dataset can be downloaded from the official website here: <https://github.com/zalando-research/fashion-mnist>. We provide a starter code for loading and splitting the dataset in the notebook.

⁴<https://pytorch.org/get-started/locally/>

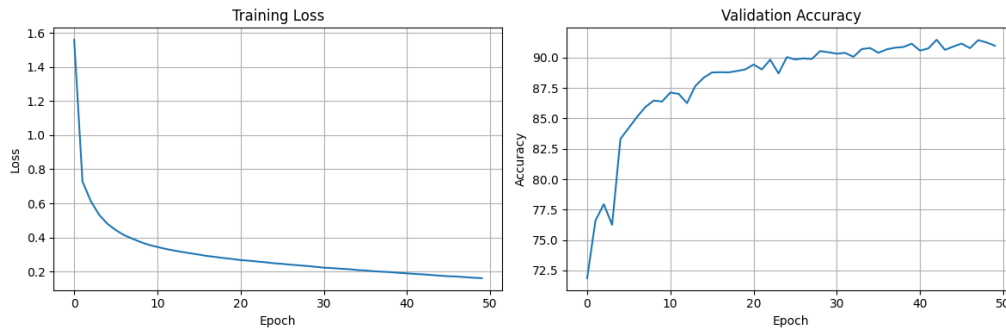


Figure 31: Training loss and test accuracy for basic CNN

Note: If you achieve more than 85% accuracy on the test set, move on to the next subproblem.

- (b) Now, try to improve your model's performance by including additional architecture elements. You should implement *at least two* of the following:
- dropout
 - batch normalization
 - data augmentation
 - different optimizers (other than vanilla stochastic gradient descent)

Try to improve your accuracy over the model in 2(a). Your new model should perform at least as well as your previous model.

TODO:

- Describe your new model additions and explain your design choices (e.g. I added dropout because...).

1. Adam Optimizer

Replacement: Changed from vanilla SGD to Adam optimizer

Rationale: Adam adapts learning rates for each parameter using momentum and adaptive learning rates, providing faster convergence and better handling of sparse gradients. It's more robust than SGD for complex loss landscapes and requires less manual tuning of learning rate schedules.

2. Batch Normalization

Addition: Added BatchNorm layers after linear transformations but before activation functions

Rationale: Batch normalization reduces internal covariate shift by normalizing layer inputs, allowing for higher learning rates and better gradient flow. It acts as a regularizer and improves training stability, making the network less sensitive to weight initialization and potentially reducing overfitting.

3. Dropout

Addition: Incorporated dropout layers after activation functions

Rationale: Dropout prevents feature co-adaptation by randomly disabling neurons during training, forcing the network to learn redundant representations. This serves as an effective regularization technique that reduces overfitting and improves generalization by creating an ensemble effect within a single model.

- Report the training, validation, and test accuracy of your best model.

Epoch 50/50: Train Accuracy: 99.26%, Val Accuracy: 92.06%

Final Test Accuracy: 91.64

- Attach the plots of training loss and validation accuracy change over epochs.

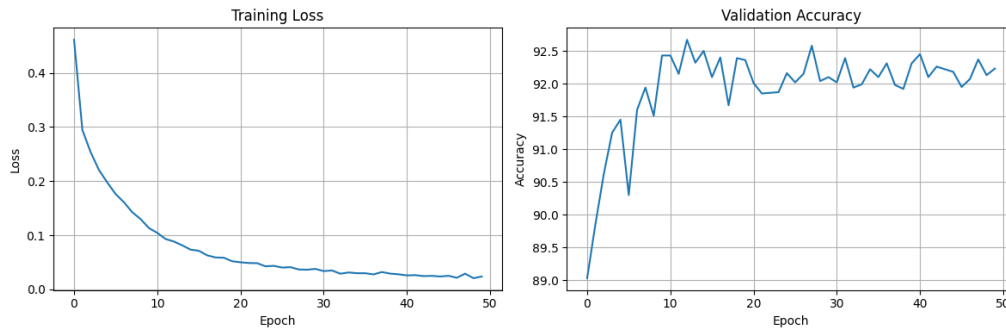


Figure 32: Training loss and test accuracy for improved CNN

- (c) Search the internet for models that others have built on FashionMNIST. This can be from research papers or even something like Kaggle. Briefly describe one technique or architecture decision which you found interesting (remember to cite your sources). It can be an extension of an existing method, and you don't need to cover the paper's results for that method. As a rough guide, anything from approximately 50-100 words of prose is sufficient.

Based on my search, one interesting technique for FashionMNIST comes from a Kaggle competition where participants used Global Average Pooling (GAP) instead of fully connected layers at the end of CNN architectures. Rather than using traditional dense layers for classification, GAP reduces each feature map to a single value by taking the spatial average, then feeds these directly to the output layer. This approach significantly reduces the number of parameters and helps prevent overfitting while maintaining spatial information until the final classification stage. The technique provides a more computationally efficient alternative to fully connected layers while achieving competitive performance on FashionMNIST.

Source

<https://www.kaggle.com/code/ctrnngtrung/a-simple-resnet-model-with-tensorflow-pytorch>
The comment: "1 global average pooling layer & 2 fully connecte layers"